

Deliverable #2

SE 3A04: Software Design II – Large System Design

Tutorial Number: T03

Group Number: G1

Group Members:

- Nadeem Elsayed
- Muhammad Ullah
- Kaitlyn Kenwell
- Aiden Sanvido
- Kurlan Beeharry

IMPORTANT NOTES

- Please document any non-standard notations that you may have used
 - Rule of Thumb: if you feel there is any doubt surrounding the meaning of your notations, document them
- Some diagrams may be difficult to fit into one page
 - Ensure that the text is readable when printed, or when viewed at 100% on a regular laptop-sized screen.
 - If you need to break a diagram onto multiple pages, please adopt a system of doing so and thoroughly explain how it can be reconnected from one page to the next; if you are unsure about this, please ask about it
- Please submit the latest version of Deliverable 1 with Deliverable 2
 - Indicate any changes you made.
- If you do NOT have a Division of Labour sheet, your deliverable will NOT be marked

1 Introduction

1.1 Purpose

This document provides a high level overview of the **SCEMAS** architecture, it includes the design considerations for the overall system along with its subsystems and how they interact with each other.

This document shall only be read by **SCEMAS** developers, project managers, domain experts, investors and government officials. **SCEMAS** Deliverable 1 is a pre-requirement for the understanding of the document, along with technical knowledge on IoT (Internet of Things) and Software Development to better perceive the design decisions made for this system.

1.2 System Description

The system follows the Presentation-Abstraction-Control (**PAC**) architecture style, with a number of subsystems that follow the Repository architecture style. The **PAC** architecture allows our system to represent various presentations of the data collected from numerous real-time data sources that collect different types of information at different frequencies. This architecture style will aid in managing information from the sensors and outputting them through multiple interfaces for the different categories of users.

A Repository architecture design was chosen for the subsystems, as it supports the access to and from data stores for different clients, and is essential to the scalable nature of the **SCEMAS** system.

The system also uses the Pipe and Filter architecture for the Sensor Data Management. This is beneficial to allow the parallel flow of sensor inputs and the processing needed when presenting or storing the data.

1.3 Overview

Section 2 presents the Analysis Class Diagram of **SCEMAS**. In Section 3, the document describes the main architecture (**PAC**) and subsystems styles (Repository, and Pipe and Filter) along with their justifications for the system, as well as the alternatives considered and why they were not suited for our system. There is also a visual representation of the main architecture showing the interactions of different parts of the system with the System Architecture Diagram. Lastly, in Section 4, the interactions and responsibilities of each class presented in the Analysis Class Diagram are represented using Class Responsibility Collaboration (CRC) Cards to better understand each of their roles in the system.

2 Analysis Class Diagram

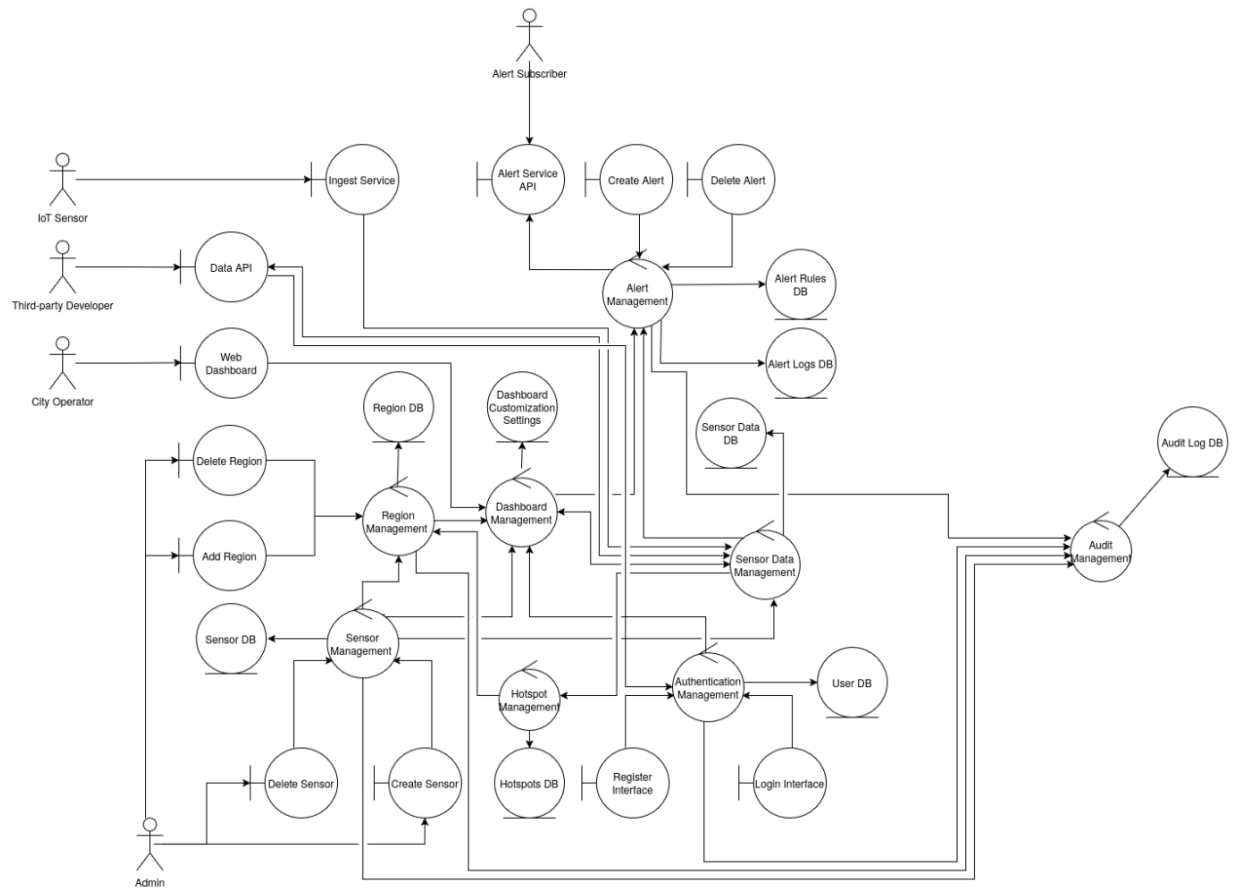


Figure 1. Analysis Class Diagram

3 Architectural Design

3.1 System Architecture

SCEMAS will utilize an interaction-oriented architecture, specifically Presentation-Abstraction-Controller as the high level architecture style of the system. The presentation corresponds with the dashboard that city operators will be interacting with, and will be layered to account for visualizations at different data rates. The controllers and abstractions correspond to various subsystems integral to managing the dashboard view, and ingesting and processing data from sensors around the service area.

The controller is broken down into several subsystems, defined as follows:

Subsystem	Purpose	Architecture Style
Sensor Management Controller	Stores raw anonymized sensor data and aggregated statistics	Repository
Sensor Data Controller	Ingest and process sensor data	Pipe and Filter
Alert Management	Records alerts for anomalous data	Repository
Account Management	Create, login, and manage user accounts	Repository
Sensor Management	Create and manage IoT sensors	Repository
Region Management	Create and manage named regions within the service area	Repository
Dashboard Management	Specify what visualizations to display on the web dashboard.	Repository
Authentication Management	Manage logins, account creation, and account deletion	Repository

Figure 2. Subsystem Architecture Table

Subsystems are elaborated upon in section 3.2.

As a high-level architecture, **PAC** is most suitable for our system given the primary concern of data visualization, and differing update rates on that data. The data visualized on the **SCEMAS** dashboard is very heterogeneous, as such, a tightly coupled, single agent architecture like Model-View-Controller is not suitable. Furthermore, the low coupling between agents in a **PAC** system, and the ability to add or remove agents as needed corresponds naturally with our system allowing operators to customize their dashboard, adding or removing visualizations over different data as required. Finally, the low coupling

between agents means that localized failures (e.g. sensors failing to report valid data for a given region) remain localized, and don't cascade into broader availability issues for the system at large.

There are several subsystems in **SCEMAS** primarily concerned with interaction-oriented data storage, both by human administrators, or as the data sink of another subsystem. We concluded that the Repository architecture style was a natural fit for these subsystems. Since these subsystems are not agents contributing to a single shared concern, we determined that Blackboard would not be a suitable architecture design. Furthermore, as these subsystems are concerned with the long-term archival/retrieval of data, rather than the real-time flow of data through a pipeline, we determined that data-flow architectures were also not appropriate.

Finally, there is a single data-flow oriented subsystem in **SCEMAS**, namely the subsystem concerned with real-time data ingress from sensors. As this subsystem is concerned with the flow and transformation of data, not with the storage of that data, data-centered architectures are not suitable. We concluded that a Pipe and Filter architecture style is most suitable for this subsystem. Real-time data ingress and processing is not a dynamic process, and can be completed with a small number of well-defined filters. Furthermore, this architecture is highly parallelizable, which ensures that the ingress service can scale with a larger number of sensors connected to a single deployment.

We considered a Batch Sequential architecture for this subsystem, and concluded that such an architecture would present too narrow a bottleneck, given the lack of well-defined data batches, and the potential for high sensor multiplicity to overwhelm the capacity of a Batch Sequential system.

Finally, the Process-Control and Master-Slave architectures are completely unsuited to the concerns of our system. Process-Control is concerned with the closed-loop control of a system variable, rather than the ingress and processing of data through a pipeline. Master-Slave is concerned with redundant slaves performing tasks orchestrated by the Master, and is also not suitable for any portion of **SCEMAS**.

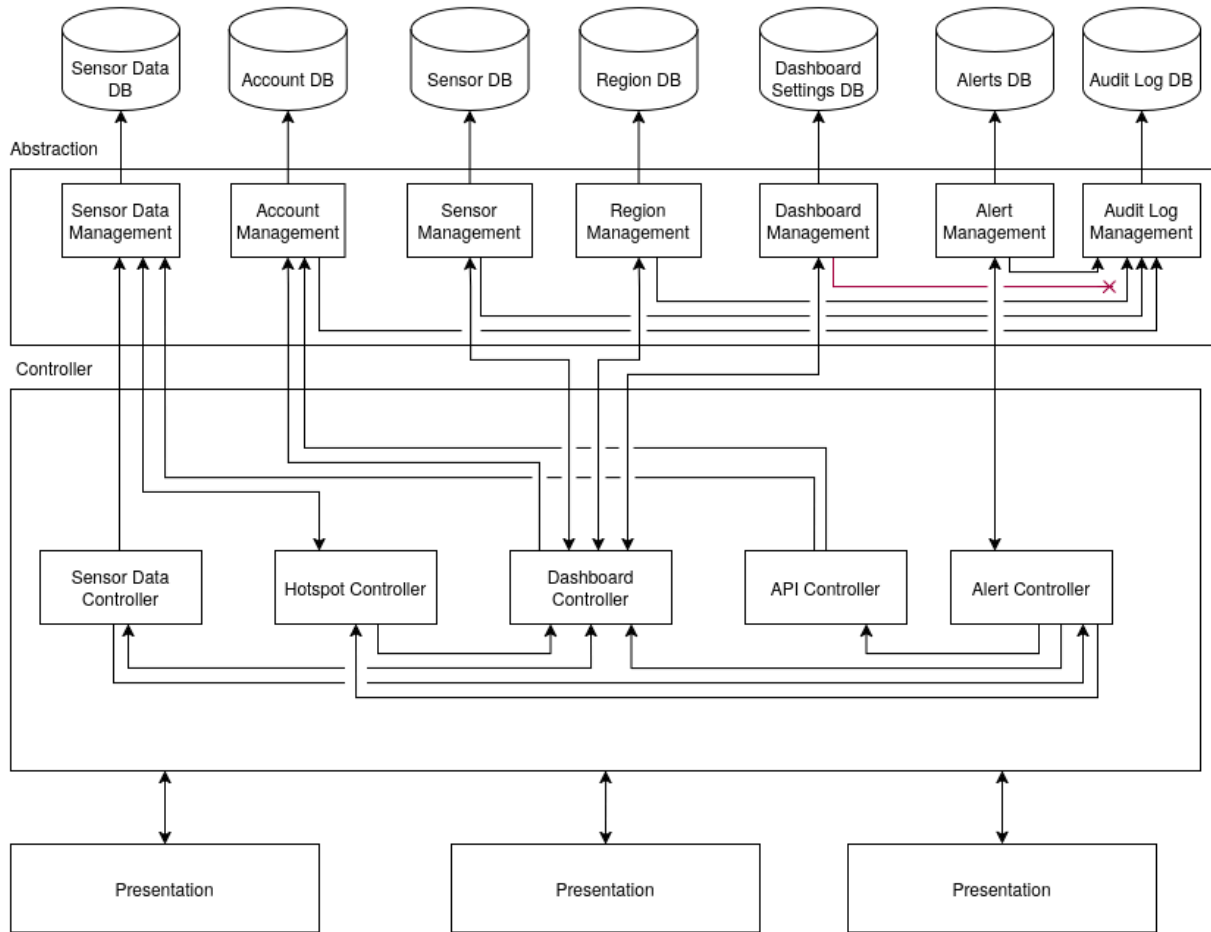


Figure 3. System Architecture Diagram

3.2 Subsystems

Our main system has been subdivided into seven distinct subsystems: Sensor Data Management, Account Management, Sensor Management, Region Management, Dashboard Management, Alert Management, and Sensor Data Controller.

The Sensor Data Management is responsible for taking in real-time data from telemetry sensors within a region and storing them into a database. The subsystem communicates to both Hotspot Controller and Sensor Data Controller which directly communicates to all the other subsystems. For Account Management, any subscribed accounts are able to view real-time data using REST API from the API Controller. For Sensor Management, every sensor will have different telemetry data assigned to it. For Region Management, certain regions will have certain readings of environmental indicators. For Dashboard Management, the readings are displayed real time for city operators to view. Lastly, for Alert Management, the real-time data is compared with threshold value if the region is hazardous.

The Account Management is responsible for managing and creating user accounts, logins, access to aggregated data, and subscribing to alert rules. Users are prompted to create an account and are able to

view data of a specific region via the API controller that only allows data within the particular region to be sent using REST API. The alert management sends alerts through the API controller to users that have subscribed for an alert.

The Sensor Management is responsible for managing and tracking IoT sensors. The sensor Management is in relationship with the Dashboard Management, Region Management, and Sensor Data Management. For Dashboard Management, the specific sensors can be removed and added by city operators. For Region Management, certain regions have only certain available IoT sensors that are registered within the service area. For Sensor Data Management telemetry, data comes in from the specified sensor using the Sensor Data Controller into the Sensor Data DB.

The Region Management is responsible for creating and managing named regions within a specified service area. The Region Management is in relationship with the Dashboard Management, Sensor Management, Account management, and Sensor Data Management. For Dashboard Management, regions can be added or removed by city operators if service boundaries were to change in an area. For Sensor Management, certain regions only have certain IoT sensors placed in the service area. For Account Management, users can be subscribed to a specific region for alert rules. For Sensor Data Management, telemetry data is obtained at a specific region.

The Dashboard Management is responsible for managing the displays of the web dashboard for city operators. The Dashboard Management is in relationship with all the specified management. For Alert Management, alerts can be added or removed by the city operator. For Region Management, regions can also be added or removed by the city operator. For Sensor Management, sensors can be added or removed. City operators have access to the sensor data from Sensor Data Management via the Sensor Data Controller which is displayed real-time.

The Alert Management is responsible for adding, changing, or deleting health hazard thresholds by the city operator. This management communicates via Dashboard Controller of a specific region of a specific sensor if the telemetry data has gone over the threshold value to the city operator. The display can be changed from the Dashboard Management, and communication is sent to the user account via the API Controller. The Alert Management also communicates with the Hotspot Controller to identify the region where there is a significant/high amount of alerts.

Sensor Data Controller is responsible for using a Pipe and Filter to aggregate data that is sent to Sensor Data DB. The data is displayed to the dashboard for the city operator and the data is also displayed to the account user. The sensor data also allows the aggregate data to be compared to the alert rules via Alert Controller.

4 Class Responsibility Collaboration (CRC) Cards

Class Name: Region Management (Controller)	
Responsibility:	Collaborators:
• Query region info • Request region(s) registered sensors • Modify existing region(s) • Register new region(s) • Log administrative actions related to regions	• Delete Region • Add Region • Sensor Management • Hotspot Controller • Audit Management • Dashboard Management • Region DB

Class Name: Delete Region (Boundary)	
Responsibility:	Collaborators:
• Handles click-event of “Delete Region” button • Display warning upon unsuccessful attempt to delete region(s)	• Region Management

Class Name: Add Region (Boundary)	
Responsibility:	Collaborators:
• Handles click-event of “Add Region” button • Handles submission of new region details • Display warning upon unsuccessful attempt to add region(s)	• Region Management

Class Name: Region DB (Entity)	
Responsibility:	Collaborators:
• Store region(s)	• Region Management

Class Name: Sensor Management (Controller)	
Responsibility:	Collaborators:
• Query sensor specifications • Query sensors registered to a particular region • Modify existing sensor(s) • Register new sensor(s) • Log administrative actions related to sensors	• Delete Sensor • Create Sensor • Sensor DB • Region Management • Dashboard Management • Sensor Data Management

	• Audit Management
--	--

Class Name: Delete Sensor (Boundary)	
Responsibility:	Collaborators:
• Handles click-event of “Delete Sensor” button • Display warning upon unsuccessful attempt to delete sensor(s)	• Sensor Management

Class Name: Create Sensor (Boundary)	
Responsibility:	Collaborators:
• Handles click-event of “Create Sensor” button • Display warning upon unsuccessful attempt to create sensor(s)	• Sensor Management

Class Name: Hotspot Management (Controller)	
Responsibility:	Collaborators:
• Track regions with sensors exceeding their defined alert thresholds • Rank hotspot regions by the frequency and severity of alert rule violations • Register new hotspot(s) • Modify existing hotspots(s) • Query hotspot(s)	• Hotspots DB • Region Management • Sensor Data Management

Class Name: Hotspots DB (Entity)	
Responsibility:	Collaborators:
• Store hotspot(s)	• Hotspot Management

Class Name: Authentication Management (Controller)	
Responsibility:	Collaborators:
• Authenticate users • Role-based authorization • Query registered user(s) • Register new user(s) • Modify existing user(s) • Log administrative actions related to account management	• Register Interface • Login Interface • Audit Management • User DB • Dashboard Management • Data API

Class Name: Register Interface (Boundary)	
Responsibility:	Collaborators:
• Handles click-event of “Register Account” button • Handles submission of account information • Display warning upon unsuccessful attempt to register user	• Authentication Management

Class Name: Login Interface (Boundary)	
Responsibility:	Collaborators:
• Handles click-event of “Login” button • Handles submission of username and password • Display warning upon unsuccessful attempt to login user	• Authentication Management

Class Name: User DB (Entity)	
Responsibility:	Collaborators:
• Stores registered user(s)	• Authentication Management

Class Name: Dashboard Management (Controller)	
Responsibility:	Collaborators:
• Updates dashboard customization settings • Requests data for selected region(s) • Requests active alert(s) • Receives live sensor data • Receives list of active sensors • Receives status of system health • Routes sensor, alert, region, and user information to the web dashboard	• Web Dashboard • Region Management • Sensor Management • Authentication Management • Sensor Data Management • Alert Management • Dashboard Customization Settings

Class Name: Dashboard Customization Settings (Entity)	
Responsibility:	Collaborators:
• Stores dashboard customization settings	• Dashboard Management

Class Name: Web Dashboard (Boundary)	
Responsibility:	Collaborators:
- Provides the user interface for the city operator - This displays regions, hotspots, sensor data, sensors, and authentication	Dashboard Management

Class Name: Sensor DB (Entity)	
Responsibility:	Collaborators:
Stores all the sensors on the network, the date they were registered, and their type	Sensor Management

Class Name: Audit Log DB (Entity)	
Responsibility:	Collaborators:
Stores logs for Alerts, Sensors, Authentication, and Regions	Audit Management

Class Name: Audit Management (Controller)	
Responsibility:	Collaborators:
- Create Sensor Management logs - Create Authentication Management logs - Create Region Management logs - Create Alert Management logs - Store logs in Audit Log DB	- Audit Log DB - Sensor Management - Region Management - Alert Management - Authentication Management

Class Name: Alert Logs DB (Entity)	
Responsibility:	Collaborators:
Store Alert logs for when a city operator does any action through alert management (ex: creating/deleting alert). Stores city operator info, date, and action	Alert Management

Class Name: Alert Rules DB (Entity)	
Responsibility:	Collaborators:
Stores all the alert rules that determine whether sensor data warrants an alert	Alert Management

Class Name: Delete Alert (Boundary)	
Responsibility:	Collaborators:
Deletes an already created alert	Alert Management

Class Name: Create Alert (Boundary)	
Responsibility:	Collaborators:
Creates an alert with an alert type and sensor levels.	Alert Management

Class Name: Alert Service API (Boundary)	
Responsibility:	Collaborators:
Sends Alerts to subscribers	Alert Management

Class Name: Alert Management (Controller)	
Responsibility:	Collaborators:
- Delete an alert - Log Alerts - Send Alert data to Audit Management - Automatically create alerts based on sensor data, region data, and alert rules - Send out alerts when user confirms through dashboard	- Alert Service API - Create Alert - Delete Alert - Alert Rules DB - Alert Logs DB - Audit Management - Sensor Data Management - Dashboard Management

Class Name: Sensor Data DB (Entity)
--

Responsibility:	Collaborators:
• Store Sensor Data	• Sensor Data Management

Class Name: Data API (Boundary)	
Responsibility:	Collaborators:
• Allows third party developers to look at Sensor Data	• Sensor Data Management • Authentication Management

Class Name: Ingest Service (Boundary)	
Responsibility:	Collaborators:
• Ingests data from sensors	• Sensor Data Management

Class Name: Sensor Data Management (Controller)	
Responsibility:	Collaborators:
• Stores Sensor Data in Sensor Data DB • Sends data to Alert Management to determine if alerts should be created • Processes data through Ingest Service • Sends data to third party developers through Data API • Displays data on Dashboard • Sends data to Hotspot Management to determine hotspots • Maps Sensor Data to Sensors through Sensor Management	• Sensor Data DB • Sensor Management • Hotspot Management • Dashboard Management • Ingest Service • Alert Management

A Division of Labour

Kurlan Beeharry

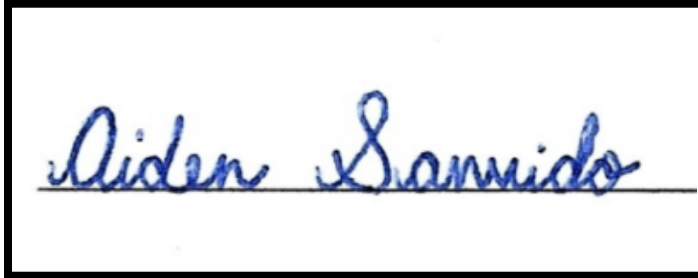
- 1.1 Purpose
- 1.2 System Description
- 1.3 Overview
- Contributed to Analysis Class Diagram
- Reviewed document
- Formatted document
 - Fonts
 - Styles
 - Page numbers
 - Spacing
 - Consistent capitalization and styles



Aiden Sanvido

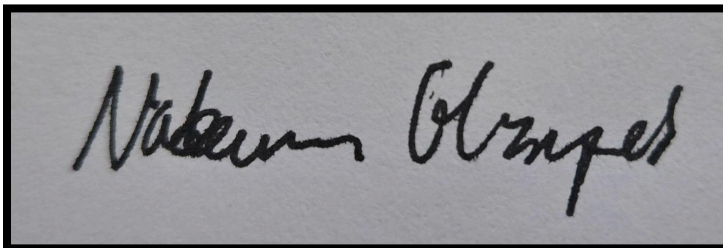
- Contributed to Analysis Class Diagram
- Worked on and finished the following CRCs in Section 4:
 - Delete Region
 - Add Region
 - Sensor DB
 - Region Management
 - Sensor Management
 - Delete Sensor
 - Create Sensor
 - Hotspot Management
 - Hotspots DB
 - Register Interface

- Authentication Management
- Login Interface
- User DB
- Dashboard Management
- Dashboard Customization Settings



Nadeem Elsayed

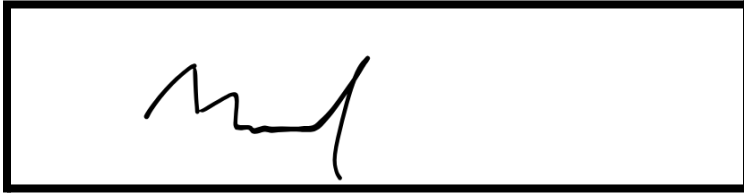
- Contributed to Analysis Class Diagram
- Worked on following CRCs in Section 4:
 - Sensor DB
 - Audit Management
 - Audit Log DB
 - Sensor Data DB
 - Alert Logs DB
 - Alert Rules DB
 - Delete Alert
 - Create Alert
 - Alert Service API
 - Alert Management
 - Sensor Data Management
 - Ingest Service
 - Data API
 - Web Dashboard



Muhammad Ullah

- Contributed to Analysis Class Diagram

- Worked on section 3.2



Kaitlyn Kenwell

- Contributed to Analysis Class Diagram
- Created section 3.1 System Architecture
- Created System Architecture Diagram

